Sécurité des communications et des services

Cryptographie asymétrique · Attaques Man-in-the-Middle · Sécurité des API REST

Public : élèves ingénieurs (2e–3e année) · **Format :** 3 modules (≈ 2 h de cours + 2 h de TD + 3 h de TP chacun) + un projet fil rouge qui se construit module après module.

🧭 Comment utiliser ce support

> **Fil conducteur : la confiance.** Le module 1 fournit les briques mathématiques de la confiance (clés publiques, signatures). Le module 2 montre ce qui se passe quand cette confiance est mal établie (l'attaquant s'invite au milieu). Le module 3 applique ces principes aux services modernes (les API REST).

Module	Thème	Question centrale
1	Cryptographie asymétrique	Comment échanger un secret avec quelqu'un que l'on n'a jamais rencontré ?
2	Attaques Man-in-the-Middle	Comment être sûr de parler à la bonne personne ?
3	Sécurité des API REST	Comment un service décide-t-il qui a le droit de faire quoi ?

Légende des niveaux de lecture : 🟢 Débutant · 🟡 Intermédiaire · 🔴 Avancé

> ⚖️ **Cadre éthique.** Tous les TP se déroulent sur des machines ou conteneurs **que vous possédez**, dans un réseau isolé. Tester une attaque sur un système sans autorisation écrite est un délit. Un ingénieur sécurité attaque pour mieux défendre.

Prérequis : Python de base, notions de réseau (IP, TCP, HTTP), ligne de commande Linux, Docker.

MODULE 1 — La cryptographie asymétrique

1. 🎣 Entrée en matière

Le fait. En 2012, des chercheurs (Heninger et al., article *Mining your Ps and Qs*) ont scanné tout Internet et collecté des millions de clés publiques TLS et SSH. Avec un simple calcul de **PGCD**, ils ont réussi à retrouver la **clé privée d'environ 0,5 % des clés RSA TLS**. La cause : des appareils (routeurs, boîtiers) générant leurs clés au démarrage, avec trop peu d'aléa. Résultat : des machines différentes partageaient un même nombre premier. Une seule division a suffi pour casser des clés censées résister à des milliards d'années de calcul.

Un second fait, plus inquiétant : « Harvest now, decrypt later ». Des acteurs étatiques peuvent enregistrer aujourd'hui du trafic chiffré pour le déchiffrer demain, quand un ordinateur quantique capable d'exécuter l'algorithme de Shor existera. Vos données de santé ou votre mémoire de recherche peuvent encore être sensibles dans 20 ans.

Et vous là-dedans ? La cryptographie asymétrique est déjà sous vos doigts :

- le cadenas du portail universitaire (TLS) ;
- votre `git push` via une clé SSH ;
- la signature qui prouve que la mise à jour de votre téléphone vient bien du constructeur ;
- les messageries chiffrées de bout en bout, les *passkeys* qui remplacent les mots de passe.

> 💥 **Question d'accroche :** si Alice et Bob ne se sont jamais rencontrés, comment peuvent-ils se mettre d'accord sur un secret **devant un public qui écoute tout** ?

---

2. 🔍 Déconstruction du concept en 3 niveaux

🟢 Niveau débutant : le cadenas ouvert et le sceau de cire

Le chiffrement. Alice fabrique des milliers de **cadenas ouverts** identiques et les distribue partout (la *clé publique*). Elle garde la **seule clé** qui les ouvre (la *clé privée*). Bob veut lui envoyer un message : il le met dans une boîte, la ferme avec un cadenas d'Alice et l'envoie. Même s'il le fait devant tout le monde, **seule Alice** peut ouvrir la boîte. Bob n'a jamais eu besoin de rencontrer Alice.

La signature. Alice possède un **sceau de cire personnel** impossible à imiter (clé privée). Tout le monde connaît son empreinte et peut vérifier qu'un courrier porte bien son sceau (clé publique). Si le courrier est modifié après coup, le sceau ne correspond plus.

> 🧠 **À retenir :** on **chiffre** avec la clé **publique** du destinataire ; on **signe** avec sa **propre clé privée**.

🟡 Niveau intermédiaire : définitions, architecture, flux

Une **paire de clés** (pk, sk) est liée mathématiquement. Connaître pk ne permet pas, en pratique, de retrouver sk.

Opération	Formule	Clé utilisée
Chiffrer	c = Enc(pk_dest, m)	publique du destinataire
Déchiffrer	m = Dec(sk_dest, c)	privée du destinataire
Signer	σ = Sign(sk_emetteur, H(m))	privée de l'émetteur
Vérifier	Verify(pk_emetteur, m, σ)	publique de l'émetteur
Échange de clés	secret = DH(sk_A, pk_B) = DH(sk_B, pk_A)	une privée + une publique

Symétrique vs asymétrique :

	Symétrique (AES, ChaCha20)	Asymétrique (RSA, ECC)
Clés	une seule, partagée	paire publique/privée
Vitesse	très rapide	100 à 1000 fois plus lent
Problème résolu	confidentialité des gros volumes	distribution des clés, identité, non-répudiation
Problème posé	comment partager la clé ?	performance, gestion de la confiance dans les clés publiques

La solution industrielle : le chiffrement hybride. On utilise l'asymétrique **uniquement** pour établir ou transporter une petite clé symétrique de session, puis tout le reste est chiffré en symétrique (AES-GCM, ChaCha20-Poly1305).

Familles d'algorithmes :

- **RSA** : repose sur la difficulté de factoriser n = p·q. Taille recommandée : ⩾ 3072 bits (≈ 128 bits de sécurité).
- **Courbes elliptiques (ECC)** : X25519 (échange de clés), Ed25519 / ECDSA (signatures). Une clé de 256 bits offre une sécurité comparable à RSA-3072.
- **Diffie-Hellman (DH/ECDH)** : accord de clé sans transport de clé.

Mini-exemple RSA (jouet, à ne jamais utiliser en vrai) :

- p = 61, q = 53 → n = 3233, φ(n) = 60 × 52 = 3120
- e = 17 (premier avec φ), d = e^{-1} mod φ = 2753
- Chiffrer m = 65 : c = 65^{17} mod 3233 = **2790**
- Déchiffrer : 2790^{2753} mod 3233 = **65** ✓

Comment RSA fonctionne, en cinq étapes :

1. Choisir deux grands nombres premiers p et q (1536 bits chacun pour une clé de 3072 bits) et calculer n = p·q.
2. Calculer φ(n) = (p−1)(q−1).
3. Choisir e premier avec φ(n). En pratique e = 65537, car il rend le chiffrement et la vérification rapides.
4. Calculer d = e^{-1} mod φ(n) avec l'algorithme d'Euclide étendu. La clé publique est (n, e) ; la clé privée est d (avec p et q pour accélérer les calculs).
5. Chiffrer : c = m^e mod n. Déchiffrer : m = c^d mod n.

Pourquoi ça marche. Comme e·d ≡ 1 (mod φ(n)), le théorème d'Euler garantit que m^(e·d) ≡ m (mod n). **Pourquoi c'est sûr :** sans factoriser n, on ne peut pas calculer φ(n), donc pas d. Factoriser un module de 3072 bits est hors de portée des algorithmes classiques connus.

L'échange de clés Diffie-Hellman avec de petits nombres. Alice et Bob choisissent publiquement un module premier p = 23 et un générateur g = 5.

Étape	Alice	Bob
Secret privé	a = 6	b = 15
Valeur publique envoyée	A = 5^6 mod 23 = **8**	B = 5^{15} mod 23 = **19**
Secret partagé	B^a = 19^6 mod 23 = **2**	A^b = 8^{15} mod 23 = **2**

Une espionne voit p, g, 8 et 19. Pour retrouver a ou b, elle doit résoudre un **logarithme discret**, un problème difficile avec de grands nombres. ⚠️ Ce protocole **n'authentifie personne** : un attaquant qui s'interpose mène deux échanges distincts, un avec chaque victime (module 2). D'où le besoin des signatures et des certificats.

Et la confiance dans la clé publique ? Si un attaquant me donne *sa* clé en prétendant que c'est celle d'Alice, tout s'effondre. D'où les **certificats X.509** : une autorité de certification (CA) signe le lien « ce nom ↔ cette clé publique ». Votre navigateur embarque ~150 CA racines de confiance.

Flux simplifié d'une connexion TLS 1.3 :

1. Le client envoie son **ClientHello** avec une part ECDHE éphémère (X25519).
2. Le serveur répond avec sa part ECDHE, son **certificat**, et une **signature** (CertificateVerify) prouvant qu'il possède la clé privée correspondante.
3. Le client vérifie la chaîne de certificats, le nom d'hôte, les dates, puis la signature.
4. Les deux côtés dérivent des clés symétriques (HKDF) : tout le trafic est ensuite chiffré en AES-GCM ou ChaCha20-Poly1305.

Anatomie d'un certificat X.509 :

Champ	Rôle	Ce que le client vérifie
Subject / SAN	nom(s) de domaine couverts	le nom demandé figure dans le SAN
Issuer	autorité qui a signé	chaîne jusqu'à une racine de confiance
Validity (notBefore / notAfter)	période de validité	la date courante est dans l'intervalle
Public Key	clé publique du serveur (RSA, ECDSA, Ed25519)	sert à vérifier la signature CertificateVerify
Key Usage / Extended Key Usage	usages autorisés (serverAuth, etc.)	cohérence avec le contexte d'emploi
Basic Constraints	CA:TRUE ou CA:FALSE, longueur de chaîne	un certificat final ne peut pas émettre d'autres certificats
SCT	preuve d'inscription dans les journaux Certificate Transparency	exigée par les navigateurs
Signature	signature de l'émetteur sur l'ensemble du certificat	valide avec la clé publique de l'émetteur

Pour l'observer sur un vrai site :

```
openssl s_client -connect example.org:443 -servername example.org </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

🔴 Niveau avancé : subtilités et cas limites

1. **Le RSA « de manuel » est cassé par construction.** Il est déterministe (même message → même chiffré) et malléable. On utilise toujours **OAEP** pour chiffrer et **PSS** pour signer. Historique : l'attaque de Bleichenbacher (1998) contre PKCS#1 v1.5 revient sans cesse (ROBOT, 2017) car l'oracle de padding fuit une information par les messages d'erreur.
2. **Confidentialité persistante (Forward Secrecy).** Avec un échange RSA statique, la compromission future de la clé privée du serveur déchiffre tout le trafic enregistré. Avec **ECDHE éphémère**, chaque session a ses propres clés jetables. TLS 1.3 a supprimé l'échange de clé RSA statique.
3. **Le désastre du nonce ECDSA.** ECDSA exige un nonce k unique et imprévisible par signature. Si k est réutilisé (cas historique de la PlayStation 3, 2010), la clé privée se calcule algébriquement (voir TD). Ed25519 génère k de façon déterministe à partir du message et de la clé : cette classe de bug disparaît.
4. **L'aléa est le maillon faible.** Faille Debian/OpenSSL de 2008 : seulement 32 768 clés possibles. Étude de 2012 ci-dessus : entropie insuffisante au démarrage. Un RNG défaillant ruine tout algorithme parfait.
5. **Canaux auxiliaires.** Temps d'exécution, consommation, cache : l'implémentation doit être en **temps constant**. Ne jamais coder sa propre crypto en production.
6. **Courbes elliptiques : validation des points.** Attaques par courbe invalide ou petit sous-groupe si l'on n'impose pas la validation de la clé publique reçue. X25519 est conçu pour limiter ces risques.
7. **Modèles de confiance.** PKI hiérarchique (TLS), *Web of Trust* (PGP), **TOFU** (SSH : on fait confiance à la première connexion, puis on détecte les changements). Chacun a ses failles : CA compromise (DigiNotar, voir module 2), clés jamais vérifiées, utilisateurs qui ignorent les alertes.
8. **Révocation et durée de vie.** CRL et OCSP sont lourds et peu fiables ; la tendance est aux **certificats à courte durée de vie** (90 jours, voire moins) et à l'automatisation (ACME).
9. **Menace quantique.** Shor casse RSA, DH et ECC ; Grover ne fait que diviser par ~2 la sécurité effective des clés symétriques (d'où AES-256). Le NIST a publié en 2024 les standards post-quantiques **FIPS 203 (ML-KEM)**, **FIPS 204 (ML-DSA)** et **FIPS 205 (SLH-DSA)**. Les navigateurs récents déploient déjà un échange **hybride X25519 + ML-KEM** : on combine classique et post-quantique, de sorte qu'il faille casser les deux.
10. **Gestion des clés.** Une clé privée se stocke dans un HSM, un KMS ou un enclave matériel, se fait tourner régulièrement, et ne se commit **jamais** dans Git.

Repères pour choisir ses algorithmes :

Usage	À privilégier	À éviter
Accord de clé	X25519 ou ECDH P-256 ; hybride X25519 + ML-KEM-768 pour les données à longue durée de vie	Diffie-Hellman statique, modules de moins de 2048 bits
Signature	Ed25519, ECDSA P-256, RSA-PSS d'au moins 3072 bits ; ML-DSA pour le post-quantique	RSA-1024, DSA, PKCS#1 v1.5 sans contrainte de compatibilité
Chiffrement asymétrique direct	préférer le chiffrement hybride (HPKE, RFC 9180) ; RSA-OAEP d'au moins 3072 bits si nécessaire	RSA sans padding, RSA PKCS#1 v1.5

Équivalences de sécurité (NIST SP 800-57) : RSA-3072 ≈ courbe de 256 bits ≈ 128 bits de sécurité ; RSA-7680 ≈ courbe de 384 bits ≈ 192 bits.

Check-list : les erreurs fréquentes en production :

Erreur	Conséquence	Bonne pratique
RSA sans padding ou en PKCS#1 v1.5	malléabilité, oracle de Bleichenbacher	OAEP pour chiffrer, PSS pour signer
Nonce réutilisé (AES-GCM, ECDSA)	perte de confidentialité ou de la clé privée	nonces uniques, Ed25519, bibliothèque AEAD qui gère le nonce
Clé RSA trop courte (moins de 2048 bits)	clé factorisable	au moins 3072 bits, ou courbe de 256 bits
Clé privée dans Git ou dans une image Docker	compromission totale	gestionnaire de secrets, rotation, analyse automatique des dépôts
Clés générées trop tôt au démarrage	clés prévisibles ou partagées	CSPRNG du système, attendre l'initialisation de l'entropie
Validation du certificat désactivée (`verify=False`)	Man-in-the-Middle	validation stricte de la chaîne et du nom, jamais de contournement en production

Erreur	Conséquence	Bonne pratique
Algorithme ou protocole « maison »	failles subtiles	bibliothèques auditées, primitives standard
Aucun inventaire cryptographique	impossible de migrer vers le post-quantique	recenser les usages, viser l'agilité cryptographique et l'hybride

---

3. 🎮 Activités immersives

Activité 1 : « Le prof a changé de clé » (en binôme, 10 min). Vous recevez un e-mail de votre enseignant : « Ma clé PGP a changé, voici la nouvelle, utilisez-la désormais. » Que faites-vous avant de faire confiance ? *Pistes : canal indépendant, comparaison d'empreinte, signature par l'ancienne clé.*

Activité 2 : « Faut-il taper yes ? » (en groupe, 15 min). Première connexion SSH à un serveur : *The authenticity of host … can't be established. Fingerprint … Are you sure you want to continue?* Une semaine plus tard : **WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!** Qu'est-ce que le TOFU ? À quel moment la réponse « yes » est-elle dangereuse ? Que vérifiez-vous avant d'effacer la ligne de `known_hosts` ?

Activité 3 : « Alerte rouge » (en groupe, 20 min). Un stagiaire a commité la clé privée du serveur de production sur un dépôt GitHub public, il y a 40 minutes. Établissez en équipe le plan d'action ordonné (révocation, nouvelle paire, réécriture de l'historique est-elle suffisante ?, communication, post-mortem).

Activité 4 : « Le jeu des cadenas » (en classe, 15 min, matériel : une boîte, deux cadenas à clé).

1. *Alice* distribue plusieurs cadenas **ouverts** à la classe et garde l'unique clé. *Bob* glisse un message dans la boîte, la ferme avec un cadenas d'Alice et la fait passer de main en main. *Eve* peut tout voir, mais ne peut pas ouvrir.
2. Rejouez la scène, mais *Mallory* remet à Bob **son propre cadenas** en prétendant qu'il appartient à Alice. Que se passe-t-il ?
3. Discussion : qui peut garantir que le cadenas est bien celui d'Alice ? Faites émerger le rôle d'un **tiers de confiance** qui appose un tampon (le certificat) et d'une **vérification hors canal** (comparer l'empreinte).

Objectif : comprendre que la cryptographie asymétrique résout la distribution des clés secrètes mais pose la question de l'authenticité des clés publiques.

---

4. 📝 Fiche de TD n°1 : Cryptographie asymétrique

Exercice 1 : RSA à la main. Soit p = 11, q = 13, e = 7.

1. Calculer n, φ(n) et la clé privée d.
2. Chiffrer m = 9.
3. Vérifier le déchiffrement (astuce : théorème des restes chinois).

Exercice 2 : Concevoir l'envoi d'un fichier de 1 Go. Alice veut envoyer un fichier de 1 Go à Bob avec confidentialité, intégrité et authenticité de l'émetteur. Proposer un protocole en citant les primitives. Pourquoi ne pas tout chiffrer en RSA ?

Exercice 3 : Facteur premier partagé. Deux serveurs publient $n_1 = 3233$ et $n_2 = 3599$ avec e = 17. Un attaquant calcule $\gcd(n_1, n_2)$.

1. Que trouve-t-il et que peut-il en déduire ?
2. Calculer la clé privée d_2.

Exercice 4 : Validation d'un certificat. Lister tout ce que le client doit vérifier pour accepter le certificat de `portail.univ.tn`. Mallory possède un certificat valide pour `mallory.com` : peut-elle l'utiliser pour se faire passer pour `portail.univ.tn` ?

Exercice 5 : Nonce réutilisé. Deux signatures ECDSA (r, s_1) et (r, s_2) sur des hachés h_1 et h_2 partagent le même r. Montrer comment retrouver k puis la clé privée d.

Exercice 6 : Diffie-Hellman et Man-in-the-Middle. Avec p = 23 et g = 5, Alice choisit a = 6 et Bob choisit b = 15.

1. Calculer A, B et le secret partagé.
2. Ève observe p, g, A et B. Que doit-elle résoudre pour retrouver le secret ?
3. Mallory s'interpose avec le secret m = 9. Décrire l'échange qu'elle mène avec chaque victime et calculer les deux secrets obtenus.
4. Quelle primitive du module permet d'empêcher cette attaque ?

Exercice 7 : Choisir la bonne primitive. Pour chaque besoin, proposer un algorithme et justifier : (a) signature du firmware d'un objet connecté, durée de vie 15 ans ; (b) messagerie chiffrée de bout en bout ; (c) authentification SSH des développeurs ; (d) signature des jetons d'accès d'une API consommée par plusieurs services ; (e) archivage de dossiers médicaux à garder confidentiels pendant 25 ans.

✅ Corrigé détaillé

Exercice 1.

1. n = 11 × 13 = **143**. φ(n) = 10 × 12 = **120**. d tel que 7d ≡ 1 (mod 120) : 7 × 103 = 721 = 6 × 120 + 1, donc **d = 103**.

2. c = 9^7 mod 143. $9^2 = 81$; $9^4 = 81^2 = 6561 \equiv 126$; $9^7 = 9^4 \times 9^2 \times 9 \equiv 126 \times 81 \times 9 \equiv 53 \times 9 = 477 \equiv$ **48**.
3. Restes chinois : mod 11, c ≡ 4 et d mod 10 = 3, donc $4^3 = 64 \equiv 9$. Mod 13, c ≡ 9 et d mod 12 = 7, donc $9^7 \equiv 9$. Les deux congruences donnent m ≡ 9 (mod 143) ✓.

⚠️ *Pièges :* oublier de réduire modulo à chaque étape (nombres énormes) ; confondre φ(n) et n pour calculer d ; utiliser e non premier avec φ(n).

Exercice 2. Protocole hybride : (1) Alice génère une clé AES-256 aléatoire K ; (2) chiffre le fichier en **AES-GCM** (nonce unique 96 bits) ; (3) chiffre K avec la clé publique de Bob (**RSA-OAEP**) ou la dérive via **X25519 + HKDF** ; (4) signe le tout avec sa clé privée (**Ed25519** ou RSA-PSS) ; (5) Bob vérifie la signature **avant** de déchiffrer. On évite le tout-RSA car il est lent et que RSA ne peut chiffrer que des messages plus courts que le module (ex. ⩽ 190 octets en RSA-2048/OAEP-SHA256). ⚠️ *Pièges :* réutiliser un nonce GCM avec la même clé (catastrophique) ; chiffrer sans authentifier l'émetteur (n'importe qui peut chiffrer pour Bob avec sa clé publique) ; valider la signature après le déchiffrement.

Exercice 3.

1. gcd(3233, 3599) = **61** : les deux modules partagent un facteur premier. On factorise les deux d'un coup : $n_1 = 61 \times 53$, $n_2 = 61 \times 59$. **Les deux clés privées sont compromises.**
2. $\varphi(n_2) = 60 \times 58 = 3480$. On cherche d_2 tel que $17\, d_2 \equiv 1 \pmod{3480}$. Euclide étendu : $1 = 1433 \times 17 - 7 \times 3480$, donc $\mathbf{d_2 = 1433}$ (vérification : $17 \times 1433 = 24361 = 7 \times 3480 + 1$ ✓). ⚠️ *Piège :* croire que deux clés « différentes » sont indépendantes. En pratique, on teste des millions de clés avec un algorithme de **PGCD par lots** (arbre de produits) en temps quasi linéaire.

Exercice 4. Le client vérifie : (a) la **chaîne** jusqu'à une CA racine de confiance, chaque signature étant valide ; (b) les **dates** de validité ; (c) que le **nom d'hôte** figure dans les champs SAN ; (d) les **usages de clé** autorisés (serverAuth) ; (e) l'état de **révocation** si applicable ; (f) la **signature CertificateVerify** prouvant la possession de la clé privée. Mallory ne peut pas utiliser son certificat pour `portail.univ.tn` : le nom ne correspond pas. Elle y parviendrait seulement si une CA émettait à tort un certificat à son nom, ou si le client acceptait une CA frauduleuse (module 2).

Exercice 5. Avec $s = k^{-1}(h + r \cdot d) \bmod q$: $s_1 - s_2 = k^{-1}(h_1 - h_2)$, donc $\mathbf{k = (h_1 - h_2)(s_1 - s_2)^{-1} \bmod q}$. Puis $\mathbf{d = (s_1 \cdot k - h_1) \cdot r^{-1} \bmod q}$. Deux signatures suffisent à tout perdre. ⚠️ *Piège :* penser qu'un aléa « presque » unique suffit ; un biais partiel du nonce permet déjà des attaques par réseaux euclidiens.

Exercice 6.

1. A = $5^6 \bmod 23$ = **8**, B = $5^{15} \bmod 23$ = **19**. Secret : $19^6 \bmod 23 = 8^{15} \bmod 23$ = **2**.
2. Ève doit résoudre le **logarithme discret** (trouver a tel que $5^a \equiv 8 \bmod 23$). Trivial ici (a = 6), infaisable avec un module de 2048 bits ou une courbe de 256 bits.

3. Mallory calcule M = 5^9 mod 23 = **11** et l'envoie **aux deux** en se faisant passer pour l'autre. Alice calcule 11^6 mod 23 = **9** (secret partagé avec Mallory, qui trouve 8^9 mod 23 = 9). Bob calcule 11^{15} mod 23 = **10** (Mallory trouve 19^9 mod 23 = 10). Mallory déchiffre les messages d'Alice avec 9, les relit, puis les rechiffre pour Bob avec 10. Les deux croient parler directement ensemble.
4. Il faut **authentifier l'échange** : chaque partie signe sa valeur publique avec sa clé privée, et l'autre vérifie avec un certificat valide (c'est le rôle de CertificateVerify dans TLS 1.3). ⚠️ *Piège :* croire que Diffie-Hellman, même éphémère, protège contre un attaquant actif. Il protège contre l'écoute passive uniquement.

Exercice 7.

- (a) **Ed25519** ou ECDSA P-256 aujourd'hui ; avec 15 ans de vie et une clé de vérification gravée dans l'appareil, prévoir l'**agilité cryptographique** et une signature post-quantique (ML-DSA, ou signature hybride).
- (b) **X25519** pour l'accord de clé avec un *ratchet* pour la confidentialité persistante (modèle Signal : X3DH / PQXDH et Double Ratchet), Ed25519 pour l'identité.
- (c) Clés **Ed25519**, protégées par mot de passe ou par une clé matérielle FIDO2.
- (d) **EdDSA ou ES256 / RS256** avec publication des clés publiques (JWKS) et rotation : les services ne détiennent que la clé de **vérification**, contrairement à un secret HMAC partagé partout.
- (e) Chiffrement symétrique **AES-256-GCM** des dossiers, avec clés enveloppées par un mécanisme **hybride X25519 + ML-KEM** ou un KMS, à cause du risque « harvest now, decrypt later ». ⚠️ *Pièges :* RSA-1024 « pour la compatibilité », HS256 avec un secret partagé entre dix services, oublier la rotation.

---

5. 🛠️ Fiche de TP n°1 : Manipuler la cryptographie asymétrique en Python

Objectifs : utiliser une bibliothèque éprouvée, réaliser un chiffrement hybride, puis casser des clés générées avec un facteur commun.

Installation : `python3 -m venv venv && source venv/bin/activate && pip install cryptography sympy`

Partie A : RSA-OAEP et signature RSA-PSS

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

priv = rsa.generate_private_key(public_exponent=65537, key_size=3072)
pub = priv.public_key()

msg = b'Rendez-vous a 18h'
oaep = padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
ct = pub.encrypt(msg, oaep)
assert priv.decrypt(ct, oaep) == msg

pss = padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH)
sig = priv.sign(msg, pss, hashes.SHA256())
pub.verify(sig, msg, pss, hashes.SHA256())   # lève InvalidSignature si le
message ou la signature est altéré
print('OK')
```

À faire : (1) chiffrer deux fois le même message et comparer les chiffrés ; (2) modifier un octet du message et relancer `verify` ; (3) chiffrer un message de 500 octets : que se passe-t-il ?

Partie B : chiffrement hybride X25519 + HKDF + AES-GCM

```
import os
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import hashes

def derive(shared):
    return HKDF(algorithm=hashes.SHA256(), length=32, salt=None, info=b'tp-
hybride').derive(shared)

bob_priv = X25519PrivateKey.generate()
bob_pub = bob_priv.public_key()

# Alice : clé éphémère jetable
eph = X25519PrivateKey.generate()
key = derive(eph.exchange(bob_pub))
nonce = os.urandom(12)
ct = AESGCM(key).encrypt(nonce, b'Fichier confidentiel', b'en-tete')
packet = (eph.public_key(), nonce, ct)
```

```
# Bob
eph_pub, nonce, ct = packet
key2 = derive(bob_priv.exchange(eph_pub))
print(AESGCM(key2).decrypt(nonce, ct, b'en-tete'))
```

À faire : altérer un octet de `ct` (que lève AES-GCM ?) ; ajouter une signature **Ed25519** de `packet` par Alice pour authentifier l'émetteur.

Partie C : casser deux clés RSA qui partagent un premier

```
from math import gcd
from sympy import randprime

p  = randprime(2**255, 2**256)
q1 = randprime(2**255, 2**256)
q2 = randprime(2**255, 2**256)
n1, n2, e = p*q1, p*q2, 65537

found = gcd(n1, n2)
assert found == p
q1_rec = n1 // found
d1 = pow(e, -1, (found-1)*(q1_rec-1))
m = 42
assert pow(pow(m, e, n1), d1, n1) == m
print('Clé privée de n1 reconstruite sans factoriser un nombre de 512 bits')
```

Partie D : certificats et signature avec OpenSSL

```
# Paire Ed25519 et certificat auto-signé (30 jours)
openssl genpkey -algorithm ed25519 -out tp.key
openssl req -new -x509 -key tp.key -out tp.crt -days 30 -subj /CN=tp.local -
addext subjectAltName=DNS:tp.local
openssl x509 -in tp.crt -noout -text | head -n 30
openssl x509 -in tp.crt -noout -fingerprint -sha256

# Signer un fichier, vérifier, puis altérer et revérifier
echo 'contrat v1' > doc.txt
openssl pkeyutl -sign -inkey tp.key -rawin -in doc.txt -out doc.sig
openssl pkey -in tp.key -pubout -out tp.pub
openssl pkeyutl -verify -pubin -inkey tp.pub -rawin -in doc.txt -sigfile
doc.sig
echo 'contrat v2' > doc.txt
openssl pkeyutl -verify -pubin -inkey tp.pub -rawin -in doc.txt -sigfile
doc.sig
```

À faire : comparer les champs *Issuer* et *Subject* ; identifier les extensions ; noter ce que renvoie la seconde vérification.

Partie E : comparer les performances

```
openssl speed -seconds 2 rsa2048 rsa3072 ecdsap256 ed25519 ecdhx25519
```

À faire : relever pour chaque algorithme le nombre de signatures et de vérifications par seconde, et conclure sur le choix entre RSA et courbes elliptiques pour un serveur qui signe beaucoup, et pour un client qui vérifie beaucoup.

👩‍🏫 Corrigé et commentaires pour l'enseignant

- **A.1** Les deux chiffrés diffèrent : OAEP est **randomisé**. Le RSA de manuel donnerait deux chiffrés identiques, ce qui fuit de l'information. **A.2** `InvalidSignature` : l'intégrité est garantie par la signature. **A.3** `ValueError: Encryption/decryption failed` : la limite est de n − 2·hLen − 2 octets, d'où l'intérêt de l'hybride.
- **B.** Altérer `ct` lève `InvalidTag` : AES-GCM est un chiffrement **authentifié** (AEAD). L'en-tête (`en-tete`) est authentifié sans être chiffré. Sans signature, Bob ne sait pas **qui** a envoyé le paquet : n'importe qui connaît sa clé publique. L'extension Ed25519 règle ce point (`sk.sign(bytes)` / `pk.verify(sig, bytes)`).
- **C.** `gcd` retrouve p en microsecondes. Discussion : à l'échelle d'Internet on applique le PGCD par lots ; la parade est d'avoir un **RNG correctement initialisé** (`/dev/urandom` après amorçage, graine matérielle) et de ne jamais générer de clés trop tôt au boot.
- **D.** *Issuer* et *Subject* sont identiques : le certificat est **auto-signé**, donc aucune tierce partie ne garantit le lien nom ↔ clé. La première vérification affiche `Signature Verified Successfully`, la seconde `Signature Verification Failure` : le moindre octet modifié invalide la signature. L'empreinte SHA-256 sert à la **vérification hors canal** : on compare l'empreinte reçue à celle que l'on a calculée soi-même.
- **E.** Les valeurs dépendent de la machine, mais l'ordre de grandeur est stable : la **signature RSA-3072 est de 50 à 100 fois plus lente** qu'Ed25519, alors que sa **vérification est très rapide** (petit exposant e). Ed25519 et X25519 sont rapides dans les deux sens et leurs clés sont courtes (32 octets). Conclusion : RSA reste acceptable pour des certificats vérifiés très souvent et signés rarement, mais pour un service qui signe beaucoup (jetons, messages), les courbes elliptiques sont nettement plus économes. La génération de clés RSA est aussi bien plus coûteuse.

---

6. 🚀 Projet fil rouge : **CampusVault** (partie 1 sur 3)

Contexte. L'université veut un coffre-fort numérique où enseignants et étudiants déposent des documents sensibles (sujets d'examen, notes, rapports de stage). Vous construisez ce système en trois étapes : cette partie établit **le noyau cryptographique**.

Cahier des charges :

1. **Génération des identités** : chaque utilisateur possède une paire X25519 (chiffrement) et une paire Ed25519 (signature), stockées dans un trousseau local **protégé par mot de passe** (dérivation Argon2id ou scrypt, chiffrement AES-GCM).
2. **Chiffrement hybride signé** : un document est chiffré pour un ou plusieurs destinataires (clé de contenu aléatoire, enveloppée pour chacun), puis signé.
3. **Vérification d'empreintes** : commande permettant de comparer l'empreinte (hash) d'une clé publique par un canal tiers ; stockage des clés de contacts en mode TOFU avec alerte en cas de changement.
4. **Rotation et révocation** : commande de rotation de clé et fichier de révocation signé.
5. **Tests** : au moins 10 tests unitaires, dont altération du chiffré, mauvaise clé, signature invalide et rejeu de nonce.

Livrables : dépôt Git (code + README), rapport de 4 pages (choix cryptographiques justifiés, modèle de menace, limites), démonstration de 5 minutes.

Barème (/20) :

Critère	Points
Justesse cryptographique (primitives, paramètres, nonces, aucun algorithme maison)	6
Gestion des clés (trousseau, rotation, révocation, TOFU)	4
Qualité du code et des tests	4
Modèle de menace et rapport	4
Démonstration et esprit critique sur les limites	2

Séances suggérées : S1 conception et trousseau · S2 chiffrement et signature · S3 contacts, rotation, tests · S4 soutenance.

Jalons détaillés :

Séance	Objectif	Livrable intermédiaire
S1	Modèle de menace, choix des primitives justifiés, trousseau chiffré par mot de passe	schéma d'architecture, module trousseau testé
S2	Chiffrement hybride signé pour un ou plusieurs destinataires	commandes `encrypt`, `decrypt`, `verify` et tests d'altération
S3	Contacts en mode TOFU, empreintes, rotation et révocation	commandes `contacts`, `rotate`, `revoke` et tests associés
S4	Finalisation, rapport, soutenance	dépôt propre, rapport de 4 pages, démonstration de 5 minutes

Pistes d'extension (bonus) : échange hybride X25519 + ML-KEM avec la bibliothèque liboqs ; affichage de l'empreinte en QR code pour une vérification en présentiel ; format de fichier versionné dont l'en-tête est authentifié ; protection du trousseau par une clé matérielle FIDO2.

(Les modules 2 et 3 suivent ci-dessous.)

MODULE 2 — Les attaques Man-in-the-Middle (MitM)

1. 🎣 Entrée en matière

Le fait. Été 2011 : la CA néerlandaise **DigiNotar** est compromise. Les attaquants émettent des certificats frauduleux, dont un pour `*.google.com`. Ces certificats, acceptés par tous les navigateurs puisque signés par une CA de confiance, servent à intercepter silencieusement les connexions Gmail de centaines de milliers d'internautes iraniens (le rapport d'enquête de Fox-IT évoque environ 300 000 adresses IP). Aucun avertissement dans le navigateur, un cadenas vert, et pourtant quelqu'un lisait tout. DigiNotar fait faillite en septembre 2011 : **une seule intrusion a détruit une entreprise entière** et poussé à créer la *Certificate Transparency*.

Et vous là-dedans ?

- Le Wi-Fi du café nommé `Cafe_Wifi` ou `Cafe_Wifi_Free` : lequel est légitime ? Un attaquant peut créer le second en 30 secondes (*evil twin*).
- Le portail captif qui demande vos identifiants universitaires pour « accéder à Internet ».
- Le QR code collé sur un parking qui mène à un faux site de paiement.

- Le message « Ce site n'est pas sécurisé, continuer quand même ? » que vous avez déjà cliqué sans lire.

> 💥 **Question d'accroche :** si le trafic est chiffré, comment un attaquant peut-il encore s'interposer ? Et comment votre navigateur sait-il qu'il parle au vrai serveur ?

---

2. 🔍 Déconstruction du concept en 3 niveaux

🟢 Niveau débutant : le facteur indélicat

Alice écrit à Bob. Le courrier passe par un facteur. Un **facteur malhonnête** ouvre l'enveloppe, lit la lettre, éventuellement la modifie, la remet dans une nouvelle enveloppe, et la livre à Bob. Ni Alice ni Bob ne remarquent quoi que ce soit : *la lettre arrive*. C'est exactement cela, un MitM : **l'attaquant ne casse rien, il s'invite sur le chemin**.

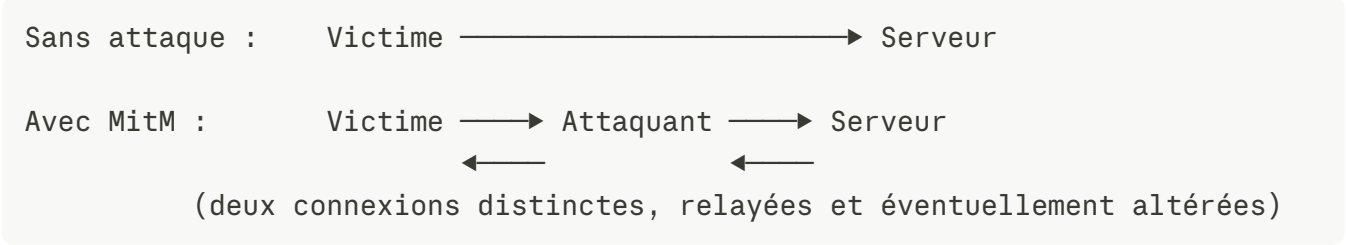
Le seul moyen de déjouer le facteur : un **sceau** que lui ne peut pas imiter (la signature du module 1) *et* un moyen sûr de reconnaître le vrai sceau de Bob (le certificat). Sans cela, vous pouvez chiffrer autant que vous voulez : vous chiffrez peut-être **pour l'attaquant**.

> 🧠 **À retenir :** le chiffrement protège contre l'écoute. Seule l'**authentification** protège contre l'interposition.

🟡 Niveau intermédiaire : deux phases, plusieurs techniques

Un MitM se déroule en deux temps :

1. **Interception** : se placer sur le chemin de communication.
2. **Exploitation** : lire (passif) ou modifier / injecter (actif), en relayant le trafic pour que la victime ne remarque rien.



Techniques d'interception courantes :

Technique	Couche	Principe	Contre-mesures principales
ARP spoofing / poisoning	2 (LAN)	l'attaquant répond à des requêtes ARP en se faisant passer pour la passerelle	Dynamic ARP Inspection, tables ARP statiques, segmentation, surveillance (arpwatch)
Rogue AP / Evil twin	2 (Wi-Fi)	faux point d'accès avec le même SSID	WPA3, 802.1X avec validation du certificat RADIUS, VPN
DHCP spoofing	3	faux serveur DHCP qui impose passerelle et DNS	DHCP snooping
DNS spoofing / poisoning	7	réponses DNS falsifiées	DNSSEC, DNS over TLS/HTTPS, résolveurs de confiance
BGP hijacking	3 (Internet)	annonce de routes frauduleuses	RPKI, filtrage des préfixes
Proxy malveillant / CA racine installée	7	logiciel ou entreprise qui installe sa propre CA	contrôle des magasins de certificats, Certificate Transparency
Compromission d'un équipement (routeur, box)	tous	l'attaquant contrôle le chemin	mises à jour, durcissement, chiffrement de bout en bout

Pourquoi HTTPS protège (en général). Pour lire une session TLS, l'attaquant devrait présenter un certificat **valide pour le nom demandé**, signé par une CA de confiance du client, et prouver qu'il possède la clé privée associée. Il ne peut pas : le chiffrement tient. Il reste donc quatre voies : **faire tomber le chiffrement** (stripping, downgrade), **obtenir un faux certificat** (CA compromise), **faire accepter un certificat invalide** (utilisateur qui clique sur « continuer »), ou **installer sa propre CA** chez la victime.

🔴 Niveau avancé : attaques sophistiquées et défenses d'ingénieur

1. **SSL stripping (2009).** L'attaquant réécrit les liens `https://` en `http://` dans les réponses d'un site visité en HTTP. Défense : **HSTS** (`Strict-Transport-Security: max-age=31536000; includeSubDomains; preload`). Limite : la **toute première visite** reste vulnérable (TOFU) d'où la **liste de préchargement HSTS** intégrée aux navigateurs.
2. **Attaques par rétrogradation (*downgrade*).** POODLE (2014), FREAK, Logjam : on force le client et le serveur à négocier un protocole ou une suite faibles. Défenses : désactiver SSLv3/TLS 1.0/1.1 et les suites export, TLS 1.3 avec protection contre la rétrogradation, **TLS_FALLBACK_SCSV**.
3. **Faux certificats et CA compromises.** Parades en profondeur : **Certificate Transparency** (journaux publics et append-only, SCT exigés par les navigateurs), enregistrements DNS **CAA** (qui peut émettre pour mon domaine), surveillance des journaux CT pour ses propres domaines. Le *pinning* de clé public (HPKP) a été abandonné car trop dangereux opérationnellement ; on lui préfère CT, et le pinning applicatif contrôlé dans les applications mobiles.
4. **CA locale installée (inspection TLS).** Cas légitimes (proxy d'entreprise, outils de débogage comme mitmproxy) et cas malveillants (adware Superfish sur des PC Lenovo, 2015, qui installait une CA racine avec une clé privée commune à toutes les machines). Ce qui change tout : **qui contrôle le magasin de certificats contrôle la confiance**.
5. **Wi-Fi d'entreprise.** Un client WPA2-Enterprise qui **ne valide pas le certificat RADIUS** se connecte à un evil twin et lui remet ses identifiants (souvent hachés en MSCHAPv2, cassables). Défense : EAP-TLS (certificats clients), validation stricte du CA et du nom du serveur RADIUS. Rappel KRACK (2017) : une faille du protocole WPA2 lui-même.
6. **Détournement BGP.** Avril 2018 : un détournement de routes visant les DNS d'Amazon Route 53 a redirigé des utilisateurs d'un service de portefeuille crypto vers un faux site, avec environ 150 000 $ de vol. Le MitM peut se jouer à l'échelle d'Internet. Défense : RPKI / ROA, filtrage de préfixes.
7. **Adversary-in-the-Middle (AiTM) contre l'authentification multifacteur.** Des *reverse proxies* de phishing relaient en direct la page légitime et volent le **cookie de session** après la saisie du code MFA (SMS, OTP, notification push). Défense efficace : **FIDO2 / WebAuthn / passkeys**, car la signature est liée à l'origine (nom de domaine) : sur un faux domaine, le navigateur ne la produit tout simplement pas.
8. **Relais d'authentification (NTLM relay).** L'attaquant relaie en temps réel un échange d'authentification vers un autre serveur. Défense : signature SMB/LDAP obligatoire, *channel binding*, abandon de NTLM.
9. **Détection.** `arpwatch` , Zeek, Suricata (règles ARP / TLS anormal), surveillance des journaux CT, tableaux de bord de changements d'empreinte SSH, alertes sur renouvellement inattendu de certificat.

---

3. 🎮 Activités immersives

Activité 1 : « Deux Wi-Fi, un seul vrai » (groupe de 4, 15 min). Dans la gare, vous voyez `Gare_Free_WiFi` et `Gare_Free_WiFi_2`. Le second ouvre un portail qui vous demande votre e-mail universitaire *et son mot de passe*. Quels indices vous alertent ? Que faites-vous concrètement (VPN, hotspot personnel, vérification auprès du personnel) ? Que perd l'attaquant si vous n'utilisez que des sites en HTTPS avec HSTS ?

Activité 2 : « Ce n'est qu'un avertissement » (débat, 10 min). Sur le portail de scolarité, le navigateur affiche `NET::ERR_CERT_AUTHORITY_INVALID`. Un camarade dit : « Clique sur *Avancé* puis *Continuer*, ça marche toujours. » Rédigez en trois phrases pourquoi c'est dangereux. Quand l'avertissement est-il un faux positif (certificat expiré, horloge de la machine) ? Comment vérifier ?

Activité 3 : « Alerte SOC » (jeu de rôles, 20 min). Vous êtes l'équipe sécurité d'un campus. L'outil de supervision signale : *l'adresse MAC de la passerelle (192.168.10.1) a changé deux fois en une minute sur le VLAN étudiants.* Listez en ordre vos actions : confirmer, localiser le port ou le point d'accès, contenir, communiquer, durcir le réseau. Que mettez-vous en place pour que ça ne se reproduise pas ?

---

4. 📝 Fiche de TD n°2 : Man-in-the-Middle

Exercice 1 : Classer les scénarios. Pour chaque cas, nommer la technique, la couche touchée et une contre-mesure :

- (a) Les utilisateurs d'un LAN reçoivent une mauvaise passerelle par défaut après l'arrivée d'un appareil inconnu.
- (b) Un site `http://banque.example` affiche soudain un formulaire de connexion en HTTP alors qu'il était en HTTPS la veille.
- (c) Le résolveur DNS du FAI renvoie une IP inattendue pour un domaine connu.
- (d) Après installation d'un « logiciel de téléchargement », tous les sites s'ouvrent sans alerte mais le certificat affiché est émis par une CA inconnue.

Exercice 2 : Lire une capture ARP. Voici trois extraits d'une capture sur un LAN (réseau 192.168.1.0/24) :

```
t=0.0s   192.168.1.1  is-at  aa:aa:aa:aa:aa:01   (réponse à une requête de
.20)
t=12.0s  192.168.1.1  is-at  bb:bb:bb:bb:bb:66   (réponse non sollicitée
envoyée à .20)
t=12.0s  192.168.1.20 is-at  bb:bb:bb:bb:bb:66   (réponse non sollicitée
envoyée à .1)
```

Que se passe-t-il ? Qui est l'attaquant ? Quelles sont les conséquences pour le trafic entre .20 et .1 ?

Exercice 3 : Ce que le client vérifie. Un attaquant passif enregistre une session HTTPS. Pourquoi ne peut-il pas la lire ? Énumérer les quatre conditions sous lesquelles un attaquant actif peut néanmoins y parvenir.

Exercice 4 : HSTS. Un site renvoie `Strict-Transport-Security: max-age=31536000; includeSubDomains`. (a) Que signifie max-age en jours ? (b) Quelle fenêtre de vulnérabilité subsiste ? (c) Quel rôle a l'option `preload` ?

Exercice 5 : Concevoir. Proposer l'architecture de sécurité réseau du Wi-Fi d'une école : authentification, chiffrement, segmentation, protections L2/L3, DNS, supervision.

✅ Corrigé détaillé

Exercice 1.

- (a) **DHCP spoofing** (couche 3/application DHCP) → *DHCP snooping* sur les commutateurs, ports de confiance.
- (b) **SSL stripping** (couche 7) → **HSTS** + preload ; vérifier que le site force HTTPS dès la première requête.
- (c) **DNS spoofing / cache poisoning** (couche 7) → DNSSEC, DoT/DoH, résolveur de confiance.
- (d) **CA racine malveillante installée localement** → désinstaller le logiciel, nettoyer le magasin de certificats, réinstaller si doute, restreindre les droits d'installation.

Exercice 2. C'est de l'**ARP poisoning** : la machine d'adresse MAC `bb:…:66` envoie des réponses ARP **non sollicitées** qui associent à la fois l'IP de la passerelle (.1) et celle de la victime (.20) à sa propre MAC. Les deux côtés lui envoient leurs paquets : **MitM bidirectionnel**. L'attaquant relaie ensuite (IP forwarding) pour que tout continue à fonctionner. Indices : la même IP change de MAC en cours de capture, et deux IP distinctes partagent une même MAC. ⚠️ *Pièges :* conclure trop vite à une attaque quand un changement de MAC est légitime (bascule de passerelle redondante HSRP/VRRP, remplacement matériel) ; ne pas regarder la **direction** des réponses (sollicitée ou non).

Exercice 3. Un attaquant passif ne voit que du trafic chiffré : sans la clé de session (dérivée par ECDHE, jamais transmise), il ne lit rien. Un attaquant actif réussit si : (1) il **présente un certificat valide pour le nom** (CA compromise ou mal-émission) ; (2) il fait **accepter un certificat invalide** à l'utilisateur ; (3) il a **installé sa CA** dans le magasin de confiance de la victime ; (4) il évite TLS par **stripping / downgrade** quand le client ou le serveur l'autorisent encore.

Exercice 4. (a) 31 536 000 s = **365 jours**. (b) La **première visite** (avant que le navigateur ait vu l'en-tête) et toute visite après expiration du max-age restent exposées au stripping. (c) `preload` demande l'inscription du domaine dans la **liste intégrée aux navigateurs** : HTTPS est forcé même à la première visite. C'est difficile à annuler, ne le demander qu'une fois **tous** les sous-domaines en HTTPS.

Exercice 5 (barème indicatif). WPA3-Enterprise ou WPA2-Enterprise avec **802.1X / EAP-TLS**, certificat serveur RADIUS **validé** par les clients ; **VLAN** distincts (étudiants, personnel, invités, IoT) avec filtrage inter-VLAN ; **isolation des clients** sur les SSID invités ; **DHCP snooping** et **Dynamic ARP Inspection** sur les commutateurs ; résolveurs **DNS** internes avec DoT/DoH et filtrage ; **détection** de rogue AP (WIDS), journalisation centralisée ; recommandation VPN pour les accès hors campus ; sensibilisation des utilisateurs. ⚠️ *Piège :* choisir un mot de passe Wi-Fi partagé (WPA2-PSK) pour tout le monde : un attaquant connaissant la clé peut déchiffrer et usurper.

---

5. 🛠️ Fiche de TP n°2 : Laboratoire MitM isolé (Docker)

> ⚠️ **Rappel éthique et technique.** Ce TP se fait **uniquement** dans le réseau Docker `internal` décrit ci-dessous, sur vos propres conteneurs. N'exécutez jamais ces commandes sur un réseau partagé (campus, domicile d'un tiers, Wi-Fi public). Le comportement du pont réseau peut varier selon l'hôte Docker (Linux natif recommandé).

Objectifs : (A) observer un empoisonnement ARP et ses effets sur du trafic HTTP en clair ; (B) le détecter avec un script ; (C) comprendre ce qu'apporte HTTPS et ce qu'implique l'installation d'une CA ; (D) auditer l'en-tête HSTS d'un site.

Mise en place

Arborescence : `tp2/docker-compose.yml` et `tp2/lab/Dockerfile`.

```
# tp2/docker-compose.yml
services:
  server:
    image: nginx:alpine
```

```
    networks:
      lab: { ipv4_address: 172.30.0.10 }
  victim:
    build: ./lab
    command: sleep infinity
    networks:
      lab: { ipv4_address: 172.30.0.20 }
  attacker:
    build: ./lab
    command: sleep infinity
    cap_add: [NET_ADMIN, NET_RAW]
    sysctls:
      - net.ipv4.ip_forward=1
    networks:
      lab: { ipv4_address: 172.30.0.66 }
networks:
  lab:
    driver: bridge
    internal: true
    ipam:
      config:
        - subnet: 172.30.0.0/24
```

```
# tp2/lab/Dockerfile
FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
    dsniff tcpdump iproute2 iputils-ping wget curl python3 python3-scapy ca-
certificates \
    && rm -rf /var/lib/apt/lists/*
```

Lancement : `cd tp2 && docker compose up -d --build`

Partie A : observer l'empoisonnement ARP

```
# 1) La victime apprend la vraie MAC du serveur, et envoie une requête avec
des identifiants fictifs en HTTP clair
docker compose exec victim sh -c 'ping -c1 172.30.0.10 && ip neigh show'
docker compose exec victim sh -c 'while true; do wget -qO-
"http://172.30.0.10/?user=etudiant&pass=secret123" >/dev/null; sleep 1; done'
&

# 2) L'attaquant empoisonne les deux sens et observe le trafic HTTP
docker compose exec attacker sh -c 'arpspoof -i eth0 -t 172.30.0.20
172.30.0.10 & arpspoof -i eth0 -t 172.30.0.10 172.30.0.20 & tcpdump -A -n -i
eth0 tcp port 80'

# 3) Pendant l'attaque, constatez le changement dans la table ARP de la
victime
docker compose exec victim ip neigh show
```

À observer : la MAC associée à `172.30.0.10` chez la victime devient celle de l'attaquant ; la requête avec `pass=secret123` apparaît **en clair** dans `tcpdump`. Arrêtez l'attaque (Ctrl+C), attendez quelques secondes et refaites `ip neigh show`.

Partie B : détecter l'attaque

Créer `arpwatch.py` dans la victime (`docker compose cp arpwatch.py victim:/arpwatch.py`) :

```
from scapy.all import sniff, ARP

table = {}

def check(pkt):
    if pkt.haslayer(ARP) and pkt[ARP].op == 2:      # is-at
        ip, mac = pkt[ARP].psrc, pkt[ARP].hwsrc
        if ip in table and table[ip] != mac:
            print(f'[ALERTE] {ip} change de MAC : {table[ip]} -> {mac}')
        table.setdefault(ip, mac)

sniff(filter='arp', prn=check, store=False)
```

Lancer le script **avant** l'attaque (`docker compose exec victim python3 /arpwatch.py`), puis relancer la partie A. Défense complémentaire : entrée ARP statique `docker compose exec victim ip neigh replace 172.30.0.10 lladdr <MAC_REELLE> dev eth0 nud permanent`, puis refaire l'attaque.

Partie C : ce que HTTPS change (et ce que fait une CA ajoutée)

```
mkdir -p mitm && docker run --rm -it -p 8080:8080 -v
"$PWD/mitm:/home/mitmproxy/.mitmproxy" mitmproxy/mitmproxy mitmdump

# Dans un autre terminal (votre hôte, vos propres requêtes uniquement)
curl -x http://localhost:8080 https://example.org -sS -o /dev/null
# 1) échec de vérification du certificat
curl -x http://localhost:8080 --cacert mitm/mitmproxy-ca-cert.pem
https://example.org -sS -o /dev/null   # 2) succès
```

À observer : le premier appel échoue (certificat non reconnu : curl code 60). Le second réussit et `mitmdump` affiche la requête déchiffrée : **dès qu'une CA est de confiance, le proxy lit tout**. Conclusion : le MitM TLS n'est pas un problème de chiffrement mais de **confiance dans les certificats**.

Partie D : auditer HSTS

```
curl -sI https://www.example.com | grep -i strict-transport-security
curl -sI http://www.example.com | head -n 5      # le site redirige-t-il vers
HTTPS ?
```

À faire : pour trois sites de votre choix, noter la présence de HSTS, la valeur de max-age, `includeSubDomains`, et conclure sur le risque de stripping.

👩‍🏫 Corrigé et commentaires pour l'enseignant

- **A.** `arpspoof` envoie en boucle des réponses ARP forgées vers la victime et le serveur ; avec `ip_forward=1` le trafic continue à passer par l'attaquant. Les identifiants fictifs apparaissent dans `tcpdump -A` car HTTP n'est pas chiffré. *Point pédagogique :* l'attaque ne casse aucun chiffrement, elle exploite le fait que **ARP n'a aucune authentification**.
- **B.** L'alerte se produit au premier paquet forgé qui diffère de la MAC apprise. *Limites à discuter :* faux positifs (basculement de passerelle), nécessité d'un état initial sain, contournement si l'attaquant agit avant le démarrage du script. Production : Dynamic ARP Inspection sur le commutateur, qui valide les réponses ARP via la table DHCP snooping.
- **C.** Le code d'erreur 60 de curl correspond à la vérification de certificat. Le proxy fabrique à la volée un certificat pour le site demandé, signé par sa propre CA. Sans cette CA dans le magasin, l'imposture est détectée ; avec elle, elle réussit. C'est exactement la logique de l'inspection TLS d'entreprise, de Superfish et des CA compromises.

- **D.** Attendre des valeurs d'au moins 6 mois (idéalement 1 an), `includeSubDomains`, et un site qui redirige 301 de HTTP vers HTTPS. L'absence de HSTS ouvre la voie au stripping lors du premier accès.

---

6. 🚀 Projet fil rouge : **CampusVault** (partie 2 sur 3)

Contexte. Le noyau cryptographique de la partie 1 existe. Il faut maintenant le faire communiquer **sur un réseau potentiellement hostile**. Vous bâtissez un **canal sécurisé** entre un client et un serveur CampusVault, puis démontrez qu'un MitM échoue.

Cahier des charges :

1. **Mini-PKI** avec OpenSSL : une CA racine locale, un certificat serveur (avec SAN), un certificat client, avec dates de validité courtes. Document de politique d'émission d'une page.
2. **Serveur TLS 1.3 uniquement** (module `ssl` de Python) avec **authentification mutuelle (mTLS)** : refuse tout client sans certificat valide, suites modernes seulement, journalisation des échecs.
3. **Client** qui valide la chaîne et le nom d'hôte, et **épingle l'empreinte** de la CA (pinning applicatif contrôlé).
4. **Scénarios d'attaque en laboratoire** (réutiliser le lab du TP) : (a) écoute passive, (b) proxy avec faux certificat, (c) tentative de downgrade vers TLS 1.2 / 1.0. Preuves à l'appui (captures, journaux) que le canal tient.
5. **Supervision** : script de détection d'ARP spoofing (partie B) intégré, et alerte en cas de changement d'empreinte de certificat.
6. **Mesure** : coût de la poignée de main TLS (avec et sans reprise de session) sur 1000 connexions.

Indices : `openssl req -x509 -newkey ed25519 -nodes -days 30 -keyout ca.key -out ca.crt -subj /CN=CampusVault-CA` pour la CA ; `ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)` avec `minimum_version = ssl.TLSVersion.TLSv1_3` et `verify_mode = ssl.CERT_REQUIRED`.

Livrables : dépôt Git (scripts de la PKI, serveur, client, lab), rapport de 4 à 6 pages (modèle de menace, résultats des attaques, limites), démonstration vidéo ou live de 5 minutes.

Barème (/20) :

Critère	Points
PKI correcte (SAN, durées, politique, protection des clés)	4

Critère	Points
mTLS et durcissement (TLS 1.3 seul, validation hôte, journalisation)	5
Scénarios d'attaque reproductibles avec preuves	5
Supervision et détection	3
Rapport, analyse critique, mesures	3

Séances suggérées : S1 PKI et serveur · S2 client, mTLS, pinning · S3 attaques et supervision · S4 mesures et soutenance.

(Le module 3 suit ci-dessous.)

MODULE 3 — La sécurité des API REST

1. 🎣 Entrée en matière

Le fait. En janvier 2023, T-Mobile annonce qu'un acteur malveillant a abusé d'une **API** pour extraire les données d'environ **37 millions de clients**, et ce pendant plusieurs semaines avant détection. En 2022, l'opérateur Optus (Australie) a subi une fuite touchant des millions de clients, attribuée par la presse spécialisée à un point d'accès d'API insuffisamment protégé. En 2021, des chercheurs ont aspiré l'intégralité des publications du réseau Parler simplement parce que les identifiants des messages étaient séquentiels et que l'API ne contrôlait rien. Dans les trois cas, aucun mot de passe n'a été « cassé » : l'API a **répondu** à des requêtes qu'elle n'aurait jamais dû accepter.

Et vous là-dedans ? Ouvrez l'application mobile de votre université. Elle appelle probablement `GET /api/students/1042/grades`. Que se passe-t-il si vous remplacez `1042` par `1043` ? Si la réponse contient les notes de quelqu'un d'autre, vous venez de découvrir la faille n°1 du classement OWASP des API.

> 💥 **Question d'accroche :** quand une application vous montre *vos* données, qu'est-ce qui garantit que le serveur ne montrerait pas celles d'un autre à la demande d'un script ?

---

2. 🔍 Déconstruction du concept en 3 niveaux

🟢 Niveau débutant : le restaurant et l'hôtel

Une API, c'est le **serveur d'un restaurant** : vous lisez le menu (la documentation), vous passez commande (la requête), le serveur la transmet à la cuisine (le serveur applicatif et la base de données) et vous rapporte le plat (la réponse). Le serveur doit pourtant faire deux vérifications que les clients oublient :

- **Qui êtes-vous ?** (*authentification*) : le badge ou la réservation.
- **Avez-vous le droit de demander ça ?** (*autorisation*) : vous pouvez commander pour votre table, pas pour la table 12 ni entrer en cuisine.

Analogie de l'hôtel : votre carte-clé ouvre *votre* chambre (authentification réussie). Mais si elle ouvrait aussi la chambre 305 parce qu'on a seulement changé le numéro de porte, ce serait une faille d'**autorisation**. C'est la plus fréquente et la plus grave des failles d'API.

> 🧠 **À retenir :** *s'authentifier* ne suffit jamais. Chaque requête doit être autorisée **sur la ressource précise** qu'elle vise.

🟡 Niveau intermédiaire : principes, architecture, flux

REST en bref : des **ressources** identifiées par une URL (`/students/42/grades`), manipulées par des **verbes HTTP** (GET lire, POST créer, PUT/PATCH modifier, DELETE supprimer), échangeant du JSON, **sans état côté serveur** (chaque requête porte son jeton), avec des **codes de statut** (200, 201, 400, 401 non authentifié, 403 interdit, 404, 429 trop de requêtes).

Architecture typique :

```
Client (web/mobile) ──HTTPS──► Passerelle d'API / WAF ──► Service applicatif
──► Base de données
                                (TLS, authN, quotas,        (autorisation par
(requêtes
                                 validation de schéma)       ressource,
logique)    paramétrées)
```

Flux d'authentification par jeton (simplifié) :

1. `POST /login` avec identifiants → le serveur renvoie un **jeton d'accès** (souvent un JWT, de courte durée).
2. Le client envoie ensuite `Authorization: Bearer <jeton>` à chaque requête.
3. Le serveur **vérifie** la signature, l'expiration, l'émetteur et l'audience, puis applique les règles d'**autorisation** sur la ressource demandée.

Le Top 10 OWASP API Security (édition 2023) :

#	Risque	Idée en une phrase
API1	Broken Object Level Authorization (BOLA)	accéder à l'objet d'un autre en changeant un identifiant
API2	Broken Authentication	jetons faibles, mots de passe non limités, JWT mal validés
API3	Broken Object Property Level Authorization	lire ou écrire des champs interdits (exposition excessive, *mass assignment*)
API4	Unrestricted Resource Consumption	pas de quotas : déni de service, factures cloud, coûts SMS
API5	Broken Function Level Authorization (BFLA)	un utilisateur simple appelle une fonction d'admin
API6	Unrestricted Access to Sensitive Business Flows	automatiser un parcours métier (achat de billets, création de comptes)
API7	Server Side Request Forgery (SSRF)	l'API est incitée à appeler des URL internes
API8	Security Misconfiguration	CORS laxiste, erreurs bavardes, services par défaut
API9	Improper Inventory Management	API oubliées, anciennes versions, environnements de test exposés
API10	Unsafe Consumption of APIs	confiance aveugle dans les réponses d'API tierces

Bonnes pratiques de base : TLS partout ; authentification forte et jetons à courte durée ; autorisation vérifiée côté serveur à **chaque** requête ; validation stricte des entrées (schéma JSON/OpenAPI, types, longueurs) ; requêtes SQL **paramétrées** ; limitation de débit et de taille ; messages d'erreur génériques ; journalisation sans secrets ; inventaire des API et gestion des versions.

🔴 Niveau avancé : subtilités et cas limites

1. **BOLA vs BFLA vs BOPLA.** BOLA : l'objet est celui d'un autre. BFLA : la **fonction** est réservée à d'autres rôles (`DELETE /users/…` , `/admin/…`). BOPLA : le **champ** est interdit (`is_admin` , `balance`) en lecture (renvoi de l'objet complet) ou en écriture (liaison automatique du JSON vers le modèle). Parade : DTO explicites avec **liste blanche** de champs.

2. **Un UUID n'est pas une autorisation.** Des identifiants imprévisibles réduisent l'énumération mais ne remplacent jamais un contrôle d'accès : ils fuient (logs, URL partagées, Referer).
3. **Pièges JWT.** Algorithme `none`, confusion d'algorithme RS256 → HS256 (la clé publique devient un secret HMAC si le serveur ne fixe pas l'algorithme attendu), secret HMAC faible et cassable hors ligne, absence de `exp` / `aud` / `iss`, injection via `kid`. Règle : **fixer la liste des algorithmes acceptés** et exiger les revendications. Le point faible structurel : un JWT autonome est **difficile à révoquer** avant son expiration (compromis : durée courte + jeton de rafraîchissement rotatif + liste de révocation).
4. **OAuth 2.0 / OIDC.** Pour les applications publiques, **Authorization Code + PKCE** ; validation stricte de `redirect_uri` ; `state` anti-CSRF ; pas de jeton dans l'URL ; stockage prudent côté client (cookies `HttpOnly; Secure; SameSite` pour le web). Séparer **scopes** (permissions d'application) et **autorisation métier** (droits sur l'objet).
5. **Limitation de débit.** Algorithmes : **token bucket** (rafales tolérées), **leaky bucket**, fenêtres glissantes. En production distribuée, état partagé (Redis) et clés de limitation multiples : IP, utilisateur, jeton, route. Les routes sensibles (`/login`, `/reset-password`, envoi de SMS) méritent des quotas stricts et des alertes.
6. **Flux métier et conditions de concurrence.** Deux requêtes simultanées sur `POST /coupon/redeem` ou `/transfer` peuvent passer un contrôle de solde avant la mise à jour (TOCTOU). Parades : transactions et verrous, contraintes d'unicité en base, **clés d'idempotence**.
7. **SSRF.** Une API qui récupère une URL fournie par l'utilisateur peut atteindre le service de métadonnées cloud (`169.254.169.254`) ; incident Capital One (2019) en est l'exemple classique. Parades : liste blanche de destinations, résolution DNS contrôlée, blocage des plages privées, IMDSv2.
8. **CORS et navigateurs.** `Access-Control-Allow-Origin: *` avec identifiants, ou reflet aveugle de l'en-tête `Origin`, ouvre les API à des sites tiers. Définir une liste précise d'origines.
9. **Performance vs sécurité.** Valider un JWT localement (JWKS en cache avec rotation) est rapide mais retarde la révocation ; l'introspection côté serveur est précise mais coûteuse. Mettre en cache **les clés**, pas les **décisions d'autorisation** sans invalidation.
10. **APIs fantômes et zombies.** `/v1` oubliée sans les corrections de `/v3`, environnements de test publics : maintenir un **inventaire**, des tests automatisés de sécurité en CI, une passerelle comme point d'entrée unique.

---

3. 🎮 Activités immersives

Activité 1 : « Le test du 1043 » (groupe, 15 min). Dans le cadre d'un audit **autorisé** de votre application de scolarité, vous remarquez `GET /api/students/1042/grades`. Quelles vérifications faites-vous, avec quel compte de test ? Que consignez-vous dans le rapport ? Comment rédigez-vous un signalement responsable si la faille est avérée ? (Rappel : ne jamais accéder aux données réelles d'autrui sans autorisation écrite.)

Activité 2 : « Un champ de trop » (binôme, 10 min). Le profil JSON renvoyé par l'API contient `{name, email, role, is_admin}` et le client envoie un `PUT /profile` avec l'objet complet. Un étudiant remplace `"role":"student"` par `"role":"teacher"` dans la requête. Que peut-il se passer ? Citez deux corrections côté serveur (liste blanche, DTO, contrôle du rôle sur l'écriture).

Activité 3 : « 1000 requêtes par seconde » (groupe, 20 min). Votre route `/reset-password` envoie un SMS payant. Un script la sollicite 1000 fois par seconde. Que se passe-t-il (coût, déni de service, harcèlement) ? Concevez les contre-mesures : quotas par IP *et* par numéro, délai croissant, CAPTCHA, alertes de coût, jeton à usage unique.

---

4. 📝 Fiche de TD n°3 : Sécurité des API REST

Exercice 1 : Classer. Associer chaque observation à une catégorie OWASP API 2023 :

- (a) `GET /api/invoices/8812` renvoie la facture d'un autre client.
- (b) Un utilisateur standard appelle `DELETE /api/users/55` et réussit.
- (c) `/api/v1/login` accepte 50 000 essais de mot de passe en une minute.
- (d) L'API renvoie la trace d'exception complète avec le chemin des fichiers et la version du framework.
- (e) `POST /api/fetch?url=http://169.254.169.254/latest/meta-data/` renvoie des secrets cloud.
- (f) Un ancien `/api/v1/` sans authentification fonctionne toujours.

Exercice 2 : Analyser un JWT. On reçoit un jeton dont l'en-tête décodé est `{"alg":"HS256","typ":"JWT"}` et la charge `{"sub":"1042","role":"student","iat":1760000000}`.

1. Quelles revendications essentielles manquent ?
2. Que se passe-t-il si le serveur accepte `alg: none` ?
3. Comment un attaquant peut-il tirer parti d'un secret HMAC de six lettres ?
4. Proposez la liste de contrôles de validation côté serveur.

Exercice 3 : Revue de code. Identifier les failles de ce handler :

```
@app.route('/api/users/<uid>', methods=['PUT'])
def update_user(uid):
    data = request.get_json()
    cur.execute(f"UPDATE users SET name='{data['name']}',
role='{data.get('role','student')}' WHERE id={uid}")
    return jsonify(cur.execute(f'SELECT * FROM users WHERE id=
{uid}').fetchone())
```

Exercice 4 : Token bucket. Un bucket a une capacité de 10 jetons et se remplit à 5 jetons/s. Un client envoie 25 requêtes à t = 0 puis 10 requêtes à t = 1 s. Combien sont acceptées ? Combien rejetées avec un code 429 ?

Exercice 5 : Modèle de menace STRIDE. Pour `POST /transfer` (virement entre comptes), dresser pour chaque lettre de STRIDE une menace concrète et une parade.

✅ Corrigé détaillé

Exercice 1. (a) **API1 BOLA** ; (b) **API5 BFLA** ; (c) **API2 Broken Authentication** (pas de limitation des essais) et aussi **API4** ; (d) **API8 Security Misconfiguration** ; (e) **API7 SSRF** ; (f) **API9 Improper Inventory Management**. ⚠️ *Piège :* plusieurs catégories peuvent coexister ; justifier le choix principal.

Exercice 2.

1. Manquent `exp` (expiration), `iss` (émetteur), `aud` (audience), et idéalement `jti` (identifiant unique pour révocation).
2. Avec `alg: none`, l'attaquant fabrique un jeton **sans signature** et se fait passer pour n'importe qui (`sub: 1` , `role: teacher`). Le serveur doit **imposer** la liste des algorithmes, jamais lire l'algorithme dans le jeton.
3. Un secret court se **brute-force hors ligne** à partir d'un seul jeton valide, sans interagir avec le serveur ; l'attaquant signe ensuite des jetons à sa guise. Secret : ⩾ 32 octets aléatoires, ou mieux, signature asymétrique (EdDSA/RS256) avec rotation de clés.
4. Contrôles : signature avec algorithme fixé, `exp` , `nbf` , `iss` , `aud` , présence des revendications requises, rôle/scopes, éventuelle liste de révocation (`jti`), taille maximale du jeton. ⚠️ *Pièges :* décoder sans vérifier la signature ; faire confiance au champ `role` pour une action sensible sans recontrôle côté serveur ; durée de vie de plusieurs jours.

Exercice 3. (1) **Injection SQL** (concaténation de chaînes) dans `name` et `uid` ; (2) **BOLA** : aucun contrôle que `uid` correspond à l'utilisateur authentifié ; (3) **Absence d'authentification** visible ; (4) **Mass assignment / BOPLA** : le client peut fixer `role` ; (5) **Exposition excessive** : `SELECT *` renvoie tous les champs (hash de mot de passe

compris) ; (6) pas de validation de schéma, pas de limitation de débit. Correction : décorateur d'authentification, comparaison `uid` / sujet du jeton (ou rôle autorisé), requêtes paramétrées, liste blanche `{name}`, DTO de sortie, schéma de validation.

Exercice 4. À t = 0 le bucket contient 10 jetons : **10 requêtes acceptées, 15 rejetées (429)**. À t = 1 s il s'est rechargé de 5 jetons : **5 acceptées, 5 rejetées**. Total : 15 acceptées, 20 rejetées. ⚠️ *Piège :* oublier que la capacité plafonne le stock de jetons même après une longue inactivité.

Exercice 5. Spoofing : usurpation d'identité → jetons courts, MFA pour les opérations sensibles. **T**ampering : modification du montant en transit → TLS + signature du corps pour les partenaires. **R**epudiation : contestation du virement → journaux d'audit signés et horodatés. **I**nformation disclosure : fuite des soldes → autorisation par objet et champs filtrés. **D**enial of service : saturation → quotas et limites de taille. **E**levation of privilege : appel par un rôle non autorisé → contrôle BFLA côté serveur. Ajouter la **clé d'idempotence** contre les doubles virements et la **vérification de solde transactionnelle** contre les conditions de concurrence.

---

5. 🛠️ Fiche de TP n°3 : Auditer puis corriger une API vulnérable (Flask)

> ⚠️ **Cadre.** L'application ci-dessous est volontairement vulnérable : exécutez-la **uniquement en local** (`127.0.0.1`), jamais exposée sur un réseau.

Installation : `pip install flask pyjwt flask-limiter` puis enregistrer le code ci-dessous sous `vuln_app.py`.

Partie A : l'API vulnérable

```
import sqlite3, datetime, jwt
from flask import Flask, request, jsonify, g

app = Flask(__name__)
SECRET = 'secret'            # faille : secret faible

def db():
    if 'db' not in g:
        g.db = sqlite3.connect('lab.db')
        g.db.row_factory = sqlite3.Row
    return g.db

def init():
```

```
    c = sqlite3.connect('lab.db')
    c.executescript('''
    DROP TABLE IF EXISTS users; DROP TABLE IF EXISTS grades;
    CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT, password TEXT, role
TEXT DEFAULT 'student');
    CREATE TABLE grades(id INTEGER PRIMARY KEY, user_id INT, course TEXT,
mark REAL);
    INSERT INTO users(name,password) VALUES ('alice','a123'),('bob','b123');
    INSERT INTO users(name,password,role) VALUES ('prof','p123','teacher');
    INSERT INTO grades(user_id,course,mark) VALUES (1,'Reseaux',15),
(2,'Reseaux',9);
    ''')
    c.commit(); c.close()

@app.post('/login')                                  # faille : aucune
limitation d'essais, mots de passe en clair
def login():
    d = request.get_json()
    u = db().execute('SELECT * FROM users WHERE name=? AND password=?',
(d['name'], d['password'])).fetchone()
    if not u:
        return jsonify(error='bad credentials'), 401
    token = jwt.encode({'sub': str(u['id']), 'role': u['role']}, SECRET,
algorithm='HS256')   # faille : pas d'exp
    return jsonify(token=token)

def current_user():
    h = request.headers.get('Authorization', '')
    return jwt.decode(h.removeprefix('Bearer '), SECRET, algorithms=
['HS256'])

@app.get('/api/grades/<int:uid>')                    # faille : BOLA
def grades(uid):
    current_user()
    rows = db().execute('SELECT course, mark FROM grades WHERE user_id=?',
(uid,)).fetchall()
    return jsonify([dict(r) for r in rows])

@app.put('/api/users/<int:uid>')                     # failles : BOLA + mass
assignment + nom de colonne injectable
def update(uid):
    current_user()
    for k, v in request.get_json().items():
        db().execute(f'UPDATE users SET {k}=? WHERE id=?', (v, uid))
    db().commit()
    return jsonify(ok=True)
```

```
if __name__ == '__main__':
    init()
    app.run(host='127.0.0.1', port=5000)
```

Partie B : audit (rôle attaquant sur *votre propre* laboratoire)

```
# 1) Obtenir un jeton en tant qu'Alice
TOKEN=$(curl -s -X POST localhost:5000/login -H 'Content-Type:
application/json' -d '{"name":"alice","password":"a123"}' | python3 -c
'import sys,json; print(json.load(sys.stdin)["token"])')

# 2) Lire ses propres notes, puis celles de Bob (BOLA)
curl -s localhost:5000/api/grades/1 -H "Authorization: Bearer $TOKEN"
curl -s localhost:5000/api/grades/2 -H "Authorization: Bearer $TOKEN"

# 3) Élévation de privilèges par mass assignment
curl -s -X PUT localhost:5000/api/users/1 -H "Authorization: Bearer $TOKEN" -
H 'Content-Type: application/json' -d '{"role":"teacher"}'
```

Questions : quelles sont les failles confirmées, et leur numéro OWASP ? Le jeton d'Alice reste-t-il valable un mois ? (Optionnel, hors ligne et sur **votre** jeton : vérifier qu'un secret aussi faible tombe avec un dictionnaire, par exemple `hashcat -m 16500 jeton.txt wordlist.txt`.)

Partie C : corriger

Appliquer les correctifs suivants à `vuln_app.py` (enregistrer en `secure_app.py`) :

```
import os, datetime, jwt
from functools import wraps
from flask import abort
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
from werkzeug.security import generate_password_hash, check_password_hash

SECRET = os.environ['JWT_SECRET']                    # au moins 32 octets
aléatoires, jamais dans le code
limiter = Limiter(get_remote_address, app=app, default_limits=['100/minute'])
ALLOWED_FIELDS = {'name'}                            # liste blanche en
écriture

def make_token(u):
    now = datetime.datetime.now(datetime.timezone.utc)
    return jwt.encode({'sub': str(u['id']), 'role': u['role'], 'iss':
```

```
'campus-api',
                        'iat': now, 'exp': now +
datetime.timedelta(minutes=15)}, SECRET, algorithm='HS256')

def auth(f):
    @wraps(f)
    def wrapper(*a, **k):
        try:
            h = request.headers.get('Authorization', '')
            g.user = jwt.decode(h.removeprefix('Bearer '), SECRET,
algorithms=['HS256'],
                                issuer='campus-api', options={'require':
['exp', 'sub', 'iss']})
        except jwt.PyJWTError:
            abort(401)
        return f(*a, **k)
    return wrapper

def owns_or_teacher(uid):
    if int(g.user['sub']) != uid and g.user['role'] != 'teacher':
        abort(403)                                      # contrôle
d'autorisation sur l'objet
```

Puis : décorer les routes avec `@auth`, appeler `owns_or_teacher(uid)` en début de handler, **rejeter** (`abort(400)`) tout champ hors `ALLOWED_FIELDS` dans `PUT`, ajouter `@limiter.limit('5/minute')` sur `/login`, stocker les mots de passe avec `generate_password_hash` et les comparer avec `check_password_hash`, et renvoyer des erreurs génériques.

Tests de validation : rejouer la partie B : l'accès aux notes de Bob doit donner **403**, la modification de `role` **400**, la sixième tentative de connexion en une minute **429**, un jeton expiré **401**.

👩‍🏫 Corrigé et commentaires pour l'enseignant

- **Failles confirmées :** BOLA sur `/api/grades/<uid>` et `/api/users/<uid>` (API1) ; mass assignment sur `role` (API3) avec élévation de privilèges ; secret JWT faible et absence d'`exp` (API2) ; aucune limitation sur `/login` (API2/API4) ; mots de passe en clair (stockage) ; nom de colonne issu de l'entrée utilisateur (injection SQL structurelle).
- **Pourquoi la liste blanche est préférable à la liste noire :** interdire `role` oublie `is_admin`, `balance`, etc. ; autoriser explicitement `name` est sûr par défaut.
- **Point de vigilance :** `owns_or_teacher` suppose que `role` du jeton est fiable ; pour les opérations critiques, relire le rôle en base. Discuter la **durée de 15 minutes** et le jeton de rafraîchissement.

- **Aller plus loin en classe :** générer un schéma OpenAPI et lancer un fuzzing de contrat (par exemple Schemathesis) ; scanner avec OWASP ZAP en mode baseline sur la version vulnérable puis corrigée, et comparer les rapports.

---

6. 🚀 Projet fil rouge : **CampusVault** (partie 3 sur 3)

Contexte. Le noyau cryptographique (partie 1) et le canal sécurisé (partie 2) existent. Il reste à exposer CampusVault comme **service** : une API REST permettant aux utilisateurs de déposer, partager et récupérer des documents **chiffrés de bout en bout**, sans que le serveur ne voie jamais le contenu en clair.

Cahier des charges :

1. **API REST** documentée en **OpenAPI** : `POST /auth/login`, `POST /documents`, `GET /documents/{id}`, `POST /documents/{id}/share`, `DELETE /documents/{id}`, `GET /audit`.
2. **Authentification** par jeton court (15 min) + jeton de rafraîchissement rotatif ; le secret ou la clé de signature est injecté par l'environnement.
3. **Autorisation à trois niveaux** : objet (propriétaire, destinataires), fonction (admin d'audit vs utilisateur), champ (liste blanche en lecture et écriture). Aucune route sans contrôle explicite.
4. **Limitation de débit** par IP, par utilisateur et par route, avec quotas renforcés sur `/auth/login` et `/share`.
5. **Validation** stricte du schéma (types, tailles maximales de fichier), requêtes paramétrées, en-têtes de sécurité, CORS restreint, erreurs génériques.
6. **Réutilisation** des parties 1 et 2 : le client chiffre et signe avant envoi ; l'API est servie en **mTLS / TLS 1.3**.
7. **Tests automatisés de sécurité** (pytest) couvrant au minimum : BOLA, BFLA, mass assignment, jeton expiré, algorithme falsifié, dépassement de quota, injection.
8. **Modèle de menace STRIDE** et inventaire des API (y compris celles que vous choisissez de ne pas exposer).

Livrables : dépôt Git complet (code, spécification OpenAPI, tests, Dockerfile / compose), rapport d'architecture et de sécurité (6 pages), jeu de tests exécutable en CI, soutenance de 10 minutes avec démonstration d'une tentative d'attaque bloquée.

Barème (/20) :

Critère	Points
Authentification et gestion des jetons	4

Critère	Points
Autorisation objet / fonction / champ (preuves par tests)	5
Limitation de débit, validation, configuration sûre	3
Intégration du chiffrement de bout en bout et du canal mTLS	3
Tests automatisés et intégration continue	3
Modèle de menace, rapport, qualité de la soutenance	2

Séances suggérées : S1 conception OpenAPI et modèle de menace · S2 authentification et autorisation · S3 quotas, validation, intégration crypto/mTLS · S4 tests de sécurité, CI et soutenance.

🏁 Évaluation globale et ressources

Évaluation suggérée du parcours complet : TD et activités (20 %) · TP rendus (20 %) · projet CampusVault en trois parties (40 %) · examen écrit de 2 h sur scénarios (20 %).

Compétences visées en fin de parcours :

- choisir et justifier des primitives cryptographiques (chiffrement authentifié, signatures, échange de clés) et les assembler sans les réinventer ;
- raisonner en termes de **chaîne de confiance** et d'adversaire actif ;
- concevoir, auditer et durcir une API selon un référentiel reconnu ;
- mener une démarche éthique : périmètre, autorisation écrite, divulgation responsable.

Références pour approfondir :

- RFC 8446 (TLS 1.3), RFC 6797 (HSTS), RFC 8017 (PKCS#1), RFC 7519 (JWT), RFC 9700 (bonnes pratiques de sécurité OAuth 2.0) ;
- OWASP API Security Top 10 (édition 2023) et OWASP Cheat Sheet Series ;
- NIST FIPS 203, 204 et 205 (cryptographie post-quantique, 2024) ;
- Heninger et al., *Mining your Ps and Qs* (USENIX Security 2012) ;
- Documentation de la bibliothèque Python `cryptography`, du projet mitmproxy, de Scapy et de Flask.